ROOTCAUSE SERVER

04 · Mapa completo del código

Documento: 04-code-map.md **Versión analizada:** 0.2.0 **Commit:** 767a67b **Fecha del análisis:** 2026-08-26

CONTENIDO

Árbol de primer nivel
  Archivos de la raíz
crates/rootcause-core — dominio y detección
  lib.rs (59 líneas)
  models.rs (737 líneas) — contratos compartidos
  security.rs (528 líneas) — lo que el agente observó
  policy.rs (305 líneas) — umbrales versionables
  detect/ — el motor
  posture.rs (250 líneas)
  runbook.rs (389 líneas)
crates/rootcause-agent — el sensor
crates/rootcause-server — el plano de control
  Rutas registradas en api.rs::router
  Métodos públicos de storage::Database
  Migraciones
  Pruebas de integración
console/ — la consola embebida
scripts/ — utilidades y guardianes
packaging/ — integración con el sistema
.github/ — automatización
Elementos sin uso, duplicados o dudosos

> Inventario jerárquico de todo lo que contiene el repositorio: directorios, archivos, módulos, tipos, funciones, scripts, migraciones y pruebas. Para cada elemento: dónde está, de qué responde, de qué depende, quién lo usa, en qué flujo participa, su importancia y su estado aparente.
>
> **Estado aparente** usa cinco valores: `activo`, `legado`, `duplicado`, `experimental`, `no determinado`. En este repositorio **todo el código es activo**; las excepciones se señalan explícitamente.

Árbol de primer nivel

Ruta	Contenido	Importancia	Estado
`crates/`	Los tres paquetes Rust del workspace	Crítica	activo
`console/`	Consola web embebida en el binario del servidor	Alta	activo
`landing/`	Página pública del producto (GitHub Pages)	Baja	activo
`docs/`	Documentación del proyecto, ADR y requisitos	Alta	activo

Ruta	Contenido	Importancia	Estado
`packaging/`	Unidades systemd, plists de launchd, instalador Windows	Media	activo
`scripts/`	Guardianes de CI, prueba de humo, SBOM, utilidades	Alta	activo
`config/`	**Vacío y fuera del control de versiones.** `git ls-files config` no devuelve nada	Ninguna	no determinado
`.github/`	Workflows, plantillas, CODEOWNERS, Dependabot	Alta	activo
`.cargo/`	`audit.toml`: exenciones de `cargo audit` con su razón	Media	activo
`target/`	Artefactos de compilación (ignorado por Git)	—	generado

Archivos de la raíz

Archivo	Responsabilidad
`Cargo.toml`	Workspace, versión `0.2.0`, dependencias compartidas, lints y perfiles `release`/`dist`
`Cargo.lock`	Árbol de dependencias fijado; versionado para reproducibilidad
`rust-toolchain.toml`	Fija Rust `1.95.0` con `clippy` y `rustfmt`
`rustfmt.toml`	Ancho 100, 4 espacios, saltos Unix, imports y módulos reordenados
`deny.toml`	Política de cadena de suministro: licencias permitidas, orígenes, duplicados, *yanked*
`Dockerfile`	Imagen en dos etapas; binario sobre `debian:bookworm-slim`, usuario 10001
`compose.yml`	Despliegue endurecido: solo lectura, sin capacidades, puerto solo en loopback
`.env.example`	Plantilla comentada de todas las variables de entorno
`.gitleaks.toml`	Lista de permitidos estrecha para el escaneo de secretos
`.hadolint.yaml`	Configuración del linter del Dockerfile
`.markdownlint-cli2.jsonc`	Reglas de Markdown aplicadas en CI
`.editorconfig`, `.gitattributes`	Codificación y saltos de línea consistentes
`AGENTS.md`	Instrucciones para agentes de IA: misión, límites innegociables e invariantes de CI
`README.md`	Portada del proyecto
`CHANGELOG.md`	Historial de versiones
`CONTRIBUTING.md`	Cómo contribuir
`SECURITY.md`	Política de divulgación
`LICENSE`	MIT

`crates/rootcause-core` — dominio y detección

Biblioteca sin entrada/salida. 4 426 líneas de Rust (pruebas incluidas), 92 pruebas unitarias. Depende solo de `chrono`, `serde`, `serde_json` y `uuid`.

`lib.rs` (59 líneas)

Símbolo	Tipo	Responsabilidad
`PROTOCOL_VERSION`	`const &str = "1.1"`	Versión del contrato de cable que implementa este build
`SUPPORTED_PROTOCOL_VERSIONS`	`const &[&str] = ["1.0","1.1"]`	Versiones que el servidor todavía acepta

Símbolo	Tipo	Responsabilidad
`protocol_is_supported`	`fn`	Comprueba si una versión reportada se puede ingerir

Reexporta lo esencial de los seis módulos para que los consumidores importen de un solo sitio. **Usado por:** `rootcause-server` y `rootcause-agent`.

`models.rs` (737 líneas) — contratos compartidos

Símbolo	Tipo	Responsabilidad
`Platform`	enum	Familia de sistema operativo; `current()` la deduce del binario
`AssetRole`	enum	`edge-server`, `internal-server` (por omisión), `database-server`, `workstation`
`AssetRegistration`	struct	Identidad que declara el agente; `role()` y `environment()` leen las etiquetas
`MetricSample`	struct	Un ciclo de medición; `validate()` rechaza porcentajes no finitos o fuera de 0–100
`TelemetryEnvelope`	struct	Lo que el agente publica: protocolo, registro opcional, muestra y superficie opcional
`Severity`	enum	`info` … `critical`, con `rank()` para ordenar
`Category`	enum	Las seis familias; `ALL` fija el orden que renderiza la consola
`IncidentStatus`	enum	`open`, `acknowledged`, `resolved`
`Evidence`	struct	Una observación; constructores `metric()` y `fact()`
`Incident`	struct	Hallazgo deduplicado con evidencia, causa, runbook y técnicas
`IncidentCandidate`	struct	Hallazgo antes de deduplicar; `into_incident()` lo materializa
`AssetStatus`	enum	`online`, `stale`, `offline`
`AssetView`	struct	El activo tal como lo ve la consola
`TopologyNode`, `TopologyEdge`, `TopologySnapshot`	struct	Mapa de defensa por zonas
`PostureDimension`, `PostureScore`	struct	Puntuación por eje y agregada, con superficies no inspeccionadas
`ExposureEntry`, `ExposureReport`	struct	Superficie expuesta de la flota
`ThreatSource`, `ThreatReport`, `DefenseCounter`	struct	Presión sobre la autenticación y defensa del propio plano de control
`StatusResponse`, `HardeningStatus`	struct	Cifras de cabecera y cómo está configurada la instancia
`IngestResponse`, `HealthResponse`, `AuditEntry`	struct	Respuestas de ingesta, salud y auditoría

Participa en: todos los flujos. **Estado:** activo.

`security.rs` (528 líneas) — lo que el agente observó

Símbolo	Responsabilidad
`Protocol`	`tcp` / `udp`
`BindScope`	`loopback` / `private` / `public`. `classify()` deriva el alcance de la dirección y **falla hacia el caso peor**

Símbolo	Responsabilidad
`PortClass`	Familia de riesgo del puerto: administración remota, base de datos, infraestructura, texto claro, compartición, correo, web, otro
`PortProfile`, `PORT_CATALOG`	Catálogo curado de **48 puertos** con su servicio y su clase
`port_profile`, `service_name`	Búsqueda en el catálogo, con reserva `"puerto N"`
`ListeningSocket`	Socket en escucha; `endpoint()` pone corchetes a IPv6, `class()` consulta el catálogo
`RemotePeer`	Conexión establecida observada
`AuthOutcome`, `AuthEvent`	Intentos de autenticación **ya agregados** por el agente
`WatchedFile`	Huella SHA-256, tamaño, fecha y modo; `is_world_writable()`
`FirewallState`	Motor, activo, número de reglas, denegación entrante por omisión
`CollectionGap`	Superficie que el agente **no pudo** inspeccionar, con su razón
`SecuritySignals`	Todo lo anterior en un ciclo, más `distinct_peers()`, `failures_by_source()`, `exposed_listeners()`, `has_gap()`

La decisión de diseño más importante del archivo: una dirección de enlace ilegible se clasifica como `Public`. Ante evidencia ambigua el producto reporta el caso peor en vez de degradar un hallazgo en silencio.

`policy.rs` (305 líneas) — umbrales versionables

Símbolo	Responsabilidad
`ResourcePolicy`	CPU, memoria, disco, muestras sostenidas y horas de margen
`IntrusionPolicy`	Fallos por origen, usuarios barridos, orígenes distribuidos, concentración de pares, multiplicador y suelo de salida
`HygienePolicy`	Actualizaciones pendientes, desviación de reloj, intervalos de silencio
`DetectionPolicy`	Las tres anteriores más `public_allowlist`
`DetectionPolicy::validate`	Rechaza una política que no podría dispararse nunca
`DetectionPolicy::is_allowed_public`	Un servidor de borde puede publicar 80 y 443
`DetectionPolicy::exposure_severity`	**El alcance multiplica el riesgo intrínseco del servicio**
`downgrade`	Baja un escalón de severidad cuando el alcance es solo red interna

Todos los `struct` llevan `#[serde(default, deny_unknown_fields)]`: un campo mal escrito en el archivo de política es un error, no un valor ignorado.

`detect/` — el motor

Archivo	Reglas que implementa	Líneas
`mod.rs`	Catálogo `RULES` (18), `DetectionInput`, `DetectionEngine`, deduplicación y huellas	426
`exposure.rs`	`exposure.service.public`, `exposure.cleartext.protocol`	252
`intrusion.rs`	`intrusion.auth.bruteforce`, `.spray`, `.distributed`, `.success_after_burst`, `intrusion.network.fanin`, `intrusion.egress.anomaly`	544

Archivo	Reglas que implementa	Líneas
`integrity.rs`	`integrity.file.changed`, `integrity.file.permissions`	213
`hygiene.rs`	`hygiene.firewall.disabled`, `.updates.pending`, `.clock.skew`	229
`availability.rs`	`availability.agent.silence` (se invoca desde el servidor, no desde `analyze`)	147
`resource.rs`	`resource.cpu.saturation`, `.memory.pressure`, `.disk.capacity`, `.disk.runway`	347

Símbolos clave de `mod.rs`:

Símbolo	Responsabilidad
`RuleInfo`, `RULES`, `rule()`	Catálogo publicado: id, categoría, título, pregunta, techo de severidad y técnicas ATT&CK
`DetectionInput`	Registro, muestra, superficie opcional, historial, línea base de archivos e instante de recepción
`DetectionEngine`	`new()` valida la política; `analyze()` corre las cinco familias y ordena
`merge_by_fingerprint`	Un servicio en `0.0.0.0` y en `::` es un hallazgo, no dos
`fingerprint`, `techniques`	Claves de deduplicación y mapeo ATT&CK, internas al crate
`fixtures`	Datos de prueba compartidos, solo en compilación de test

`posture.rs` (250 líneas)

`compute()` recorre las seis categorías, resta puntos por hallazgo abierto según severidad, pondera cada dimensión y aplica un **techo** derivado del peor hallazgo abierto: un servidor con una base de datos expuesta no puede sacar «B» porque todo lo demás esté en orden. Adjunta siempre la lista de superficies no inspeccionadas. `compute_now()` sella con el instante actual.

`runbook.rs` (389 líneas)

`StepKind` (`inspect` / `contain` / `remediate`), `RunbookStep` y cuatro generadores por plataforma: `block_source`, `restrict_listener`, `verify_file`, `enable_firewall`. Cada paso declara si necesita privilegios y si es reversible, y el módulo incluye una prueba — `no_destructive_commands` — que **hace fallar el build** si un comando destructivo entra aquí.

`crates/rootcause-agent` — el sensor

Binario de solo lectura. 2 149 líneas de Rust (pruebas incluidas), 59 pruebas unitarias.

Archivo	Responsabilidad	Depende de	Líneas
`main.rs`	CLI, validación de transporte, registro, envío, reintento exponencial, `--dry-run`	`collector`, `net`, `reqwest`, `clap`	395
`collector.rs`	Un ciclo completo: recursos primero, superficie después	`sysinfo`, `net`, `authlog`, `integrity`, `baseline`	210
`net.rs`	Sockets en escucha y pares conectados, con un parser por plataforma	`probe`	508
`authlog.rs`	Intentos de autenticación agregados	`probe`	369
`integrity.rs`	Huellas SHA-256 de archivos vigilados	`sha2`, `tokio::fs`	201

Archivo	Responsabilidad	Depende de	Líneas
`baseline.rs`	Firewall del host y parches pendientes	`probe`	334
`probe.rs`	**El único lugar que ejecuta un comando externo**	`tokio::process`	132

Funciones destacadas:

Función	Archivo	Papel
`validate_transport`	`main.rs`	Se niega a poner el token en un HTTP remoto sin autorización explícita
`stable_agent_id`	`main.rs`	UUID v5 derivado del nombre de equipo: el mismo host conserva su identidad entre reinicios
`Collector::sample`	`collector.rs`	Métricas validadas antes de salir del proceso
`Collector::security`	`collector.rs`	Junta red, autenticación, integridad y controles básicos, y sus brechas
`parse_ss`, `parse_netstat_windows`, `parse_netstat_bsd`, `parse_proc_net`	`net.rs`	Cuatro parsers puros con fixtures propias
`parse_sshd_log`, `parse_windows_security_log`	`authlog.rs`	Agregan por servicio, origen, usuario y resultado
`parse_ufw_status`, `parse_firewalld_state`, `parse_nft_ruleset`, `parse_netsh_advfirewall`, `parse_socketfilterfw`	`baseline.rs`	Un parser por motor de firewall
`parse_apt_simulation`, `parse_dnf_security`	`baseline.rs`	Conteo de parches pendientes
`probe::ALLOWED_PROGRAMS`, `is_allowed`, `run`	`probe.rs`	Lista blanca de 11 programas, `stdin` nulo, `stderr` descartado, tiempo límite y `kill_on_drop`

Todos los parsers son funciones puras que reciben texto y devuelven estructuras. Esa separación es lo que permite probar en cualquier sistema operativo el código que interpreta la salida de herramientas de otros tres.

`crates/rootcause-server` — el plano de control

Binario más biblioteca. 3 920 líneas en `src/` más 637 de pruebas de integración; 74 pruebas unitarias más 25 de integración.

Archivo	Responsabilidad	Líneas
`lib.rs`	Expone los módulos para que las pruebas conduzcan el router sin abrir un socket	16
`main.rs`	Despacho de subcomandos, arranque, trazas, anuncio y apagado ordenado	164
`config.rs`	`Cli`, `Command`, `ServeSettings`; `validate()` y `detection_policy()`	299
`state.rs`	`Runtime`, `AppState`, `hardening()`, `record_defense()`	139
`defense.rs`	`Perimeter`: cubeta de tokens y bloqueo por dirección	317
`auth.rs`	Middleware: dirección del cliente, tasa, bloqueo, token en tiempo constante	163
`headers.rs`	CSP y ocho cabeceras de seguridad; HSTS solo sobre TLS	122
`error.rs`	`ApiError` con códigos estables; el error interno nunca filtra la causa	150

Archivo	Responsabilidad	Líneas
`api.rs`	19 rutas, ingesta, informes, exportación y métricas	1 100
`storage.rs`	Todo el SQL: activos, telemetría, incidentes, líneas base, presión, defensa, auditoría, retención	1 159
`watchdog.rs`	Silencio de agentes, poda del perímetro y retención	212
`ui.rs`	Sirve la consola embebida y protege las rutas reservadas	79

Rutas registradas en `api.rs::router`

Método y ruta	Handler	Autenticada
`GET /healthz`	`health`	No
`GET /readyz`	`ready`	No
`GET /api/v1/status`	`status`	Sí
`GET /api/v1/rules`	`rules`	Sí
`GET /api/v1/policy`	`policy`	Sí
`GET /api/v1/assets`	`list_assets`	Sí
`POST /api/v1/assets/register`	`register_asset`	Sí
`GET /api/v1/assets/{id}`	`asset_detail`	Sí
`POST /api/v1/telemetry`	`ingest_telemetry`	Sí
`GET /api/v1/incidents`	`list_incidents`	Sí
`GET /api/v1/incidents/{id}`	`incident_detail`	Sí
`POST /api/v1/incidents/{id}/status`	`change_incident_status`	Sí
`GET /api/v1/incidents/{id}/runbook`	`incident_runbook`	Sí
`GET /api/v1/exposure`	`exposure`	Sí
`GET /api/v1/threats`	`threats`	Sí
`GET /api/v1/topology`	`topology`	Sí
`GET /api/v1/audit`	`audit`	Sí
`GET /api/v1/export`	`export`	Sí
`GET /metrics`	`metrics`	Sí
(cualquier otra)	`ui::static_asset`	No

Funciones públicas de `api.rs` que existen para poder probarse por separado: `build_exposure`, `build_topology`, `render_export`, `render_metrics`.

Métodos públicos de `storage::Database`

Método	Papel en el flujo
`connect`, `ping`	Arranque y `/readyz`
`upsert_asset`, `asset_registration`, `list_assets`, `asset_heartbeats`	Inventario y vigilancia de silencio
`store_sample`, `recent_samples`	Telemetría e historial para las reglas con ventana
`store_security`, `store_posture`	Última superficie y última puntuación por activo

Método	Papel en el flujo
`file_baseline`, `update_file_baseline`	Detección de cambio de archivo exactamente una vez
`record_auth_pressure`, `threat_sources`, `total_auth_failures`	Informe de amenazas de la flota
`upsert_incident`, `auto_resolve`, `list_incidents`, `incident`, `update_incident_status`	Ciclo de vida del incidente
`open_finding_counts`, `open_findings_by_asset`	Resumen de hallazgos abiertos por categoría y severidad con `COUNT(*)` y `GROUP BY`: permite puntuar la postura de toda la flota sin materializarla
`counts`	Cifras de cabecera y métricas
`record_defense_event`, `defense_counters`	Defensa del propio plano de control
`audit`, `list_audit`	Rastro inmutable
`purge`	Retención de telemetría, presión y eventos de defensa

Migraciones

Archivo	Qué aporta
`0001_init.sql`	`assets`, `telemetry`, `incidents`, `audit_log` y sus índices
`0002_security_surface.sql`	Columnas de seguridad en `assets`, `category` en `incidents`, y las tablas `file_baselines`, `auth_pressure`, `defense_events`

Toda columna añadida en `0002` es nullable o tiene valor por omisión, para que una base creada por `0.1.0` siga funcionando tras la actualización.

Pruebas de integración

`tests/control_plane.rs` (637 líneas, 25 pruebas) conduce el router en proceso con `tower::ServiceExt`. Cubre autenticación, bloqueo, cabeceras, ingesta, auto-resolución, honestidad de cobertura, informes, auditoría, exportación y métricas.

`console/` — la consola embebida

Archivo	Responsabilidad	Líneas
`index.html`	Cascarón: navegación de 8 vistas, tablas vacías y diálogos	413
`app.js`	Toda la lógica: estado, llamadas al API, renderizado con la API del DOM	1 177
`styles.css`	Estilos, sin dependencias externas	1 152
`rootcause.svg`	Logotipo	11

Vistas: panel, superficie, amenazas, incidentes, activos, topología, reglas y sistema. Estado global en `state`; refresco cada 10 s (`REFRESH_INTERVAL_MS`).

Restricción que define su estilo: la CSP que envía el servidor no admite `script-src 'unsafe-inline'`, así que **cada nodo se construye con la API del DOM**, nunca desde una cadena. Un nombre de equipo o de proceso reportado por un agente no puede interpretarse como marcado. `scripts/guard_console.py` hace fallar el build si aparece un manejador en línea, un origen externo o una asignación a `innerHTML`.

`scripts/` — utilidades y guardianes

Script	Qué hace	Lo ejecuta
`guard_claims.py`	Deriva del código las cifras que afirman los documentos (18 reglas, versión) y exige que toda acción de GitHub esté pinneada a SHA de 40 caracteres	CI, trabajo «Coherencia de lo que se afirma»
`guard_console.py`	Prohíbe script en línea, origen externo y sumideros de marcado en la consola; verifica la CSP de `headers.rs`	CI, trabajo «Consola embebida»
`guard_encoding.py`	UTF-8 sin BOM, sin CRLF, sin mojibake, en 109 archivos de texto	CI, trabajo «Integridad del texto»
`sbom.py`	Genera un SBOM CycloneDX desde `cargo metadata`, solo con la biblioteca estándar	CI, trabajo «Inventario de dependencias»
`docs_pdf.py`	Convierte `docs/system-documentation/*.md` a HTML autocontenido y a PDF; dibuja los diagramas Mermaid con `mmdc` e imprime con Chrome, Chromium o Edge	Persona, a demanda
`smoke.sh`	Prueba de humo real: compila, arranca servidor y agente, y comprueba lo que comprobaría un operador	CI, trabajo «Prueba de humo»
`backup.sh` / `backup.ps1`	Respalda la base con `.backup` de SQLite y **verifica la copia** antes de darla por buena: integridad y recuento de filas contra el origen	Persona, o una tarea programada
`dev.sh` / `dev.ps1`	Arranca el servidor en desarrollo generando token si falta	Persona
`run-agent.sh` / `run-agent.ps1`	Ejecuta el sensor contra el plano local; sin argumentos, un solo ciclo	Persona

El SBOM se escribió dentro del repositorio a propósito: un producto de seguridad que mete una acción de terceros en su cadena de suministro **para describir su cadena de suministro** ha cambiado el problema por uno mayor.

`packaging/` — integración con el sistema

Archivo	Plataforma	Endurecimiento destacado
`linux/rootcause-server.service`	systemd	`NoNewPrivileges`, `ProtectSystem=strict`, `CapabilityBoundingSet=`, `MemoryDenyWriteExecute`, familias de socket restringidas
`linux/rootcause-agent.service`	systemd	Equivalente, para el sensor
`macos/com.rootcause.server.plist`	launchd	Servicio del plano de control
`macos/com.rootcause.agent.plist`	launchd	Servicio del sensor
`windows/rootcause-server.iss`	Inno Setup	Instalador del servidor

`.github/` — automatización

Workflow	Disparo	Trabajos
`ci.yml`	push, PR, manual	15: formato, clippy ×3 SO, pruebas ×6 combinaciones, documentación, cobertura, dependencias, MSRV, consola, afirmaciones, workflows, Markdown, codificación, contenedor, humo y **puerta final**
`security.yml`	push, PR, semanal (lunes 06:17 UTC), manual	Avisos, secretos en el histórico, dependencias del repositorio, workflows, SBOM
`codeql.yml`	—	Análisis de `javascript-typescript` y `actions`
`release.yml`	etiquetas `v*`, manual	Verificar, construir por plataforma, publicar
`deploy-landing.yml`	push a `main` en `landing/`	Publica en GitHub Pages tras comprobar que la página no carga recursos de otro origen

Otros: `dependabot.yml`, `CODEOWNERS`, `pull_request_template.md` y tres plantillas de incidencia (capacidad, defecto, falso positivo).

Elementos sin uso, duplicados o dudosos

Se buscaron activamente. El resultado es corto:

Elemento	Observación	Clasificación
`config/`	Directorio vacío **y ausente del repositorio**: 0 archivos en `git ls-files`, sin historial. Git no versiona directorios vacíos, así que quien clone el repositorio no lo verá. Ninguna referencia desde el código, los scripts, los workflows ni el `Dockerfile`	**no determinado** — resto local de la estructura inicial
`crates/rootcause-server/src/rca.rs`	**Ya eliminado** en el commit `2efedb2` por quedar huérfano tras la reescritura del motor	resuelto
`landing/rootcause.svg` y `console/rootcause.svg`	Mismo contenido en dos lugares	duplicación **intencional**: la página pública y la consola embebida se despliegan por separado y no comparten árbol de activos
`Category::parse` para `"resource"` y `default_category()`	Ambas devuelven `Category::Resource`	no es duplicación: una parsea entrada externa, la otra da compatibilidad a un incidente `0.1` sin categoría

No se encontró código muerto, funciones sin llamar ni módulos huérfanos en los tres crates.

Anterior: 03 · Arquitectura · **Siguiente:** 05 · Referencia técnica · **Índice:** README
